Advanced Probabilistic Machine Learning

Ralf Herbrich, Rainer Schlosser

Approximate Inference: Direct Message Approximation

Overview

1. Direct Message Approximation
2. Approximate Messages for Deterministic Functions
   - Product Factor
   - Leaky Rectified Linear Unit Factor
3. Application: Bayesian Neural Networks

Overview

1. **Direct Message Approximation**
2. Approximate Messages for Deterministic Functions
   - Product Factor
   - Leaky Rectified Linear Unit Factor
3. Application: Bayesian Neural Networks

Sum-Product Algorithm Approximations Revisited

- The key operation for factor $f(x_1, x_2, \dots, x_n)$ and variable X_1 is

$$m_{f \to X_1}(x_1) = \sum_{\{x_2\}} \cdots \sum_{\{x_n\}} f(x_1, x_2, \dots, x_n) \prod_{j=2}^{n} m_{X_j \to f}(x_j)$$

If all $m_{X_j \to f}(\cdot)$ are in the exponential family with sufficient statistic $\boldsymbol{T}_j(\cdot)$, the result **might not be** in the exponential family with sufficient statistic $\boldsymbol{T}_1(\cdot)$!

- Based on outgoing messages, we can compute both non-normalized marginals $p_{X_1}(\cdot)$ and $m_{X_1 \to f}(\cdot)$

$$p_{X_1}(x_1) = \prod_{f \in \mathrm{ne}(X_1)} m_{f \to X_1}(x_1) \qquad m_{X_1 \to f}(x_1) = \frac{p_{X_1}(x_1)}{m_{f \to X_1}(x_1)}$$

If all $m_{f \to X_1}(\cdot)$ are in the exponential family with sufficient statistic $\boldsymbol{T}_1(\cdot)$, so would be $p_{X_1}(\cdot)$ and all $m_{X_1 \to f}(\cdot)$!

- **Idea**:
  1. We approximate the message $m_{f \to X_1}(\cdot)$ by an exponential family $\widehat{m}_{f \to X_1}(\cdot) \propto \exp\left(\boldsymbol{\theta}_{f \to X_1}^{\mathrm{T}} \boldsymbol{T}(\cdot)\right)$
  2. We measure the approximation quality in the normalized marginal, **not** the outgoing message

$$\boldsymbol{\theta}_{f \to X_1}^{*} = \arg\min_{\boldsymbol{\theta}_{f \to X_1}} D_\alpha \left[\frac{1}{Z} \cdot m_{f \to X_1}(\cdot) \cdot m_{X_1 \to f}(\cdot), \frac{1}{\hat{Z}} \cdot \exp\left(\boldsymbol{\theta}_{f \to X_1}^{\mathrm{T}} \boldsymbol{T}(\cdot)\right) \cdot m_{X_1 \to f}(\cdot) \right]$$

True normalized marginal with $Z \coloneqq \int m_{f \to X_1}(x) \cdot m_{X_1 \to f}(x)\, \mathrm{d}x$

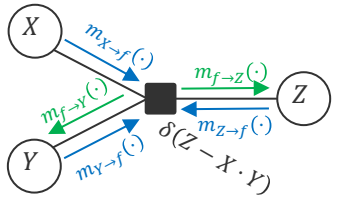
Approximate marginal with $\hat{Z} \coloneqq \int \exp\left(\boldsymbol{\theta}_{f \to X_1}^{\mathrm{T}} \boldsymbol{T}(x)\right) \cdot m_{X_1 \to f}(x)\, \mathrm{d}x$

Motivation: Product Factor

- **Product Factor**: A deterministic factor where Z is the product of two random variables X and Y

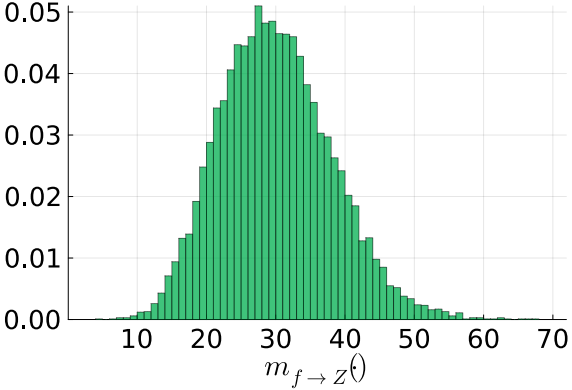
$$f(X, Y, Z) = \delta(Z - X \cdot Y)$$

  - **Example**: Recommendation model where the preference of a user to an item is modelled as the inner product of a user trait (vector) with an item trait (vector)



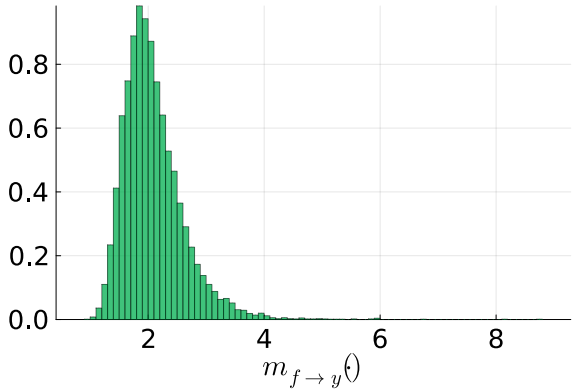
$m_{X\to f}(x) = \mathcal{N}(x; 5,1)$

$m_{Y\to f}(y) = \mathcal{N}(y; 6,1)$



$m_{X\to f}(x) = \mathcal{N}(x; 5,1)$

$m_{Z\to f}(z) = \mathcal{N}(z; 10,1)$




```julia
function sample_product_message(;
    μ_x=5.0, σ_x=1.0, μ_y=6.0, σ_y=1.0, n_samples=1000000)
    samples_msg_to_z = Vector{Float64}(undef, n_samples)
    for i in 1:n_samples
        x = randn() * σ_x + μ_x
        y = randn() * σ_y + μ_y
        samples_msg_to_z[i] = x * y
    end
end
```

```julia
function sample_division_message(;
    μ_x=5.0, σ_x=1.0, μ_z=10.0, σ_z=1.0, n_samples=1000000)
    samples_msg_to_y = Vector{Float64}(undef, n_samples)
    for i in 1:n_samples
        x = randn() * σ_x + μ_x
        z = randn() * σ_z + μ_z
        samples_msg_to_y[i] = z / x
    end
end
```

Motivation: Expectation Propagation ($\alpha = 1$)

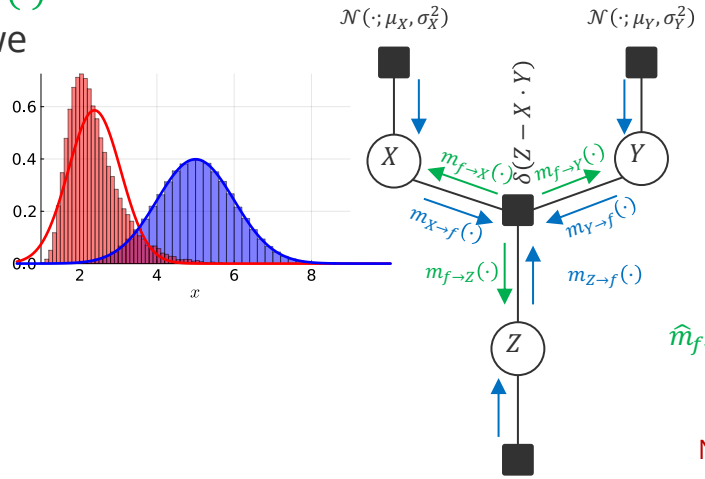
- Since the messages $m_{f\to X}(\cdot)$, $m_{f\to Y}(\cdot)$ and $m_{f\to Z}(\cdot)$ are not a Gaussian, we approximate the marginals

$$p_X(\cdot) = m_{f\to X}(\cdot)\cdot m_{X\to f}(\cdot)$$
$$p_Y(\cdot) = m_{f\to Y}(\cdot)\cdot m_{Y\to f}(\cdot)$$
$$p_Z(\cdot) = m_{f\to Z}(\cdot)\cdot m_{Z\to f}(\cdot)$$

with by virtue of

$$\hat{p}_X(\cdot) = \mathcal{N}\left(\cdot; E_{X\sim p_X}[X], \mathrm{var}_{X\sim p_X}[X]\right)$$
$$\hat{p}_Y(\cdot) = \mathcal{N}\left(\cdot; E_{Y\sim p_Y}[Y], \mathrm{var}_{Y\sim p_Y}[Y]\right)$$
$$\hat{p}_Z(\cdot) = \mathcal{N}\left(\cdot; E_{Z\sim p_Z}[Z], \mathrm{var}_{Z\sim p_Z}[Z]\right)$$

- **Idea**: Instead of algebraic integration, use importance sampling



$$\hat{m}_{f\to Y}(y) = \frac{\hat{p}_Y(y)}{m_{Y\to f}(y)} = \frac{\mathcal{N}(y;\hat{\mu}_Y,\hat{\sigma}_Y^2)}{\mathcal{N}(y;\mu_Y,\sigma_Y^2)}$$

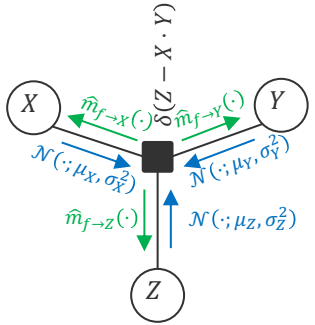
Not possible because $\hat{\sigma}_Y^2 > \sigma_Y^2$

Advanced Probabilistic Machine Learning

Unit 7 – Approximate Inference: Direct Message Approximation

Motivation: Variational Inference ($\alpha = 0$)

HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam



- Looking at the factor function $f(X,Y,Z)$ we see that

$$f(X,Y,Z) = \lim_{\epsilon^2\to 0} \frac{1}{\sqrt{2\pi\epsilon^2}} \cdot \exp\left(-\frac{1}{2\epsilon^2}(Z - X\cdot Y)^2\right)$$

$$\propto \lim_{\epsilon^2\to 0} \exp\left(-\frac{Z^2}{2}\cdot\frac{1}{\epsilon^2} + Z\cdot\frac{XY}{\epsilon^2}\right) \qquad \propto \lim_{\epsilon^2\to 0} \exp\left(-\frac{X^2}{2}\cdot\frac{Y^2}{\epsilon^2} + X\cdot\frac{ZY}{\epsilon^2}\right) \qquad \propto \lim_{\epsilon^2\to 0} \exp\left(-\frac{Y^2}{2}\cdot\frac{X^2}{\epsilon^2} + Y\cdot\frac{ZX}{\epsilon^2}\right)$$

- The variational approximations of the outgoing messages are

$\widehat{m}_{f\to Z}(z) = \mathcal{N}(z; m_Z, s_Z^2)$

$$m_Z = \frac{\tau_Z}{\rho_Z} = \lim_{\epsilon^2\to 0} \frac{E[X]\cdot E[Y]\cdot\epsilon^2}{\epsilon^2}$$

$$s_Z^2 = \frac{1}{\rho_Z} = \lim_{\epsilon^2\to 0} \epsilon^2$$

$\widehat{m}_{f\to X}(x) = \mathcal{N}(x; m_X, s_X^2)$

$$m_X = \frac{\tau_X}{\rho_X} = \lim_{\epsilon^2\to 0} \frac{E[Z]\cdot E[Y]\cdot\epsilon^2}{E[Y^2]\cdot\epsilon^2}$$

$$s_X^2 = \frac{1}{\rho_X} = \lim_{\epsilon^2\to 0} \frac{\epsilon^2}{E[Y^2]}$$

$\widehat{m}_{f\to Y}(y) = \mathcal{N}(y; m_Y, s_Y^2)$

$$m_Y = \frac{\tau_Y}{\rho_Y} = \lim_{\epsilon^2\to 0} \frac{E[Z]\cdot E[X]\cdot\epsilon^2}{E[X^2]\cdot\epsilon^2}$$

$$s_Y^2 = \frac{1}{\rho_Y} = \lim_{\epsilon^2\to 0} \frac{\epsilon^2}{E[X^2]}$$

$\widehat{m}_{f\to Z}(z) = p_Z(z) = \delta(z - \mu_X\cdot\mu_Y)$

$\widehat{m}_{f\to X}(x) = p_X(x) = \delta\left(x - \frac{\mu_Z}{\mu_Y}\right)$

$\widehat{m}_{f\to Y}(y) = p_Y(y) = \delta\left(y - \frac{\mu_Z}{\mu_X}\right)$

Massive underestimation of predictive variance reducing **all marginals** to Dirac deltas with zero uncertainty!

Direct Message Approximation

- **Challenge**: Minimizing the α-divergence *of the marginal* $p_X(\cdot)$ can lead to undefined messages ($\alpha = 1$) or messages that are too certain ($\alpha = 0$)!
- **Idea**: Instead of minimizing the α-divergence *of the marginal* $p_X(\cdot)$, minimize the α-divergence *of the message* $m_{f \to X}(\cdot)$
- **Direct Message Approximation**. Given a factor graph with m factor nodes $f_1, \dots, f_m$ and n variable nodes $X_1, \dots, X_n$ as well as n sufficient statistics $\boldsymbol{T}_1(\cdot), \dots, \boldsymbol{T}_n(\cdot)$ for the approximate marginals $p_{X_1}(\cdot), \dots, p_{X_n}(\cdot)$ the *direct message approximation* for factor f_i and variable X_j is defined by

$$\boldsymbol{\theta}^*_{f_i \to X_j} = \arg\min_{\boldsymbol{\theta}_{f \to X}} D_\alpha \left[\frac{1}{Z_m} \cdot m_{f_i \to X_j}(\cdot), \exp\left(\boldsymbol{\theta}^{\mathrm{T}}_{f \to X} \boldsymbol{T}_j(\cdot) - A_j\left(\boldsymbol{\theta}_{f \to X}\right) \right) \right]$$

True normalized message with $Z_m \coloneqq \int m_{f_i \to X_j}(x)\, \mathrm{d}x$

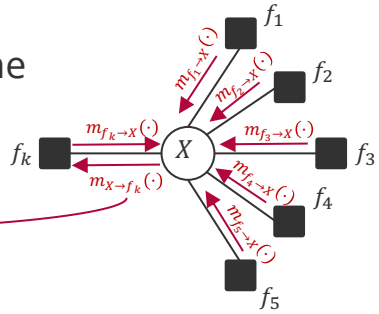
Approximate normalized message

- This is only applicable when the true message is normalizable (e.g., not for $m_{f \to X}(x) = \mathbb{I}(x \geq 0)$)
- It is identical to approximating the marginal $p_X(\cdot)$ if the incoming message $m_{X \to f}(\cdot)$ is uniform
- For $\alpha = 1$ this approximation method matches the expected natural statistic of the message

Direct Message Approximation: Interpretation

- **Recall**: The application of a message update from a factor f_k to a variable X is the application of Bayes' rule to X

$$p_X(x) = \prod_{i \in \text{ne}(X)} m_{f_i \to X}(x) = m_{f_k \to X}(x) \cdot \prod_{i \in \text{ne}(X) \setminus \{k\}} m_{f_i \to X}(x)$$



$$p_X(x) = m_{f \to X}(x) \cdot m_{X \to f}(x)$$

Posterior × Evidence $p(x|D) \times p(D)$ — Likelihood $p(D|x)$ — Prior $p(x)$

  - Direct Marginal Approximation is equivalent to approximating the posterior probability (and the normalization constant is the probability of data also known as *evidence*)
  - Message approximation is equivalent to approximating the likelihood directly!
- **Main Question**: Can we guarantee anything about α-divergence of the marginals when we approximate the messages directly?

Direct Message Approximation Bound

- **Theorem (Herbrich & Schlosser 2026)**. Given a factor graph and a factor node f connected to the variable X, we have the following for *any* normalized direct message approximation $\widehat{m}_{f\to X}(\cdot)$ of the normalized outgoing message $m_{f\to X}(\cdot)$ with $\mathrm{KL}\left[m_{f\to X}(\cdot), \widehat{m}_{f\to X}(\cdot)\right] = \epsilon$ and for any normalized message $m_{X\to f}(\cdot)$,

$$\mathrm{KL}\left[\frac{1}{Z} m_{f\to X}(\cdot)\cdot m_{X\to f}(\cdot), \frac{1}{\hat{Z}} \widehat{m}_{f\to X}(\cdot)\cdot m_{X\to f}(\cdot)\right] \leq \sqrt{2}\cdot \frac{\left\|m_{X\to f}(\cdot)\right\|_\infty}{Z}\cdot\left(\epsilon + \sqrt{\epsilon}\right)$$

Maximum value of density $m_{X\to f}(\cdot)$

KL divergence of *marginal*

Approximate Probability of data

$$\hat{Z} \coloneqq \int \widehat{m}_{f\to X}(x)\cdot m_{X\to f}(x)\,\mathrm{d}x$$

Probability of data

$$Z \coloneqq \int m_{f\to X}(x)\cdot m_{X\to f}(x)\,\mathrm{d}x$$

KL divergence of *message*

  - The probability of data is large for well-chosen models
  - The maximum value of the density $m_{X\to f}(\cdot)$ is $(2\pi)^{-\frac{1}{2}}\cdot\sigma_{X\to f}^{-1}$ for Gaussians
  - The maximum value of density and probability of data are competing: For $\sigma_{X\to f}^2 \to +\infty$ the numerator converges to zero but the denominator (i.e., probability of data) converges to 0!
  - This can also be shown for the reverse KL divergence D_0

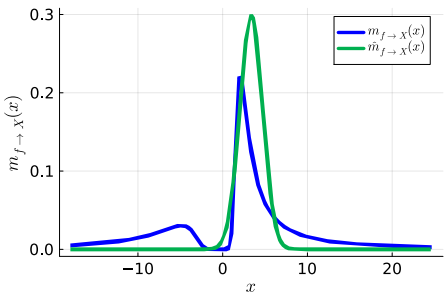
Direct Message Approximation: Example

Good Approximation

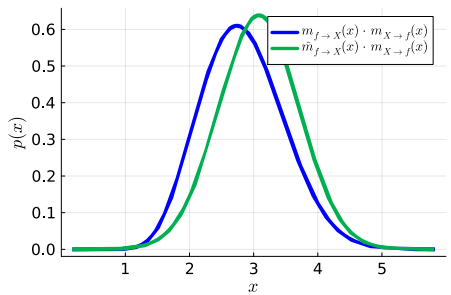
$m_{Y\to f}(y) = \mathcal{N}(y; 2, 0.5)$

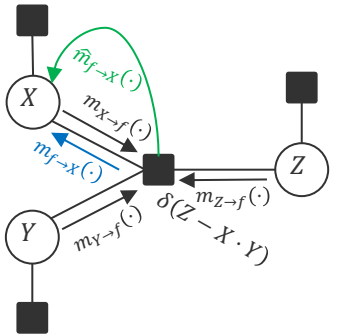
$m_{Z\to f}(z) = \mathcal{N}(z; 6, 1)$

$\hat{m}_{f\to X}(x) = \mathcal{N}(x; 3.37, 1.69)$



$m_{X\to f}(x) = \mathcal{N}(x; 3, 0.5)$



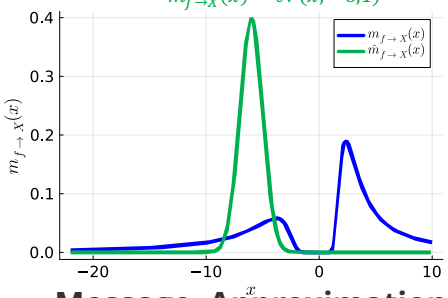


Bad Approximation

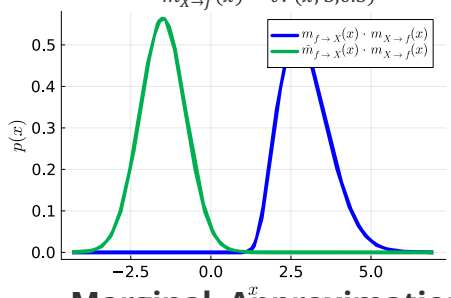
$m_{Y\to f}(y) = \mathcal{N}(y; 2, 1)$

$m_{Z\to f}(z) = \mathcal{N}(z; 10, 1)$

$\hat{m}_{f\to X}(x) = \mathcal{N}(x; -6, 1)$



$m_{X\to f}(x) = \mathcal{N}(x; 3, 0.5)$



Message Approximation | **Marginal Approximation**

Advanced Probabilistic Machine Learning

Unit 7 – Approximate Inference: Direct Message Approximation

Overview

1. Direct Message Approximation
2. **Approximate Messages for Deterministic Functions**
   - **Product Factor**
   - Leaky Rectified Linear Unit Factor
3. Application: Bayesian Neural Networks

Exponential Family Distributions: Log-Normal

- **Log-Normal Distribution**. *Given $\mu \in \mathbb{R}$ and $\sigma \in \mathbb{R}^+$, a positive continuous random variable X is said to have a Log-Normal distribution if the density is given by*

$$\mathrm{LN}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma x} \cdot \exp\left(-\frac{(\log(x) - \mu)^2}{2\sigma^2}\right) \cdot \mathbb{I}(x > 0)$$

- **Properties**:

$$X \sim \mathrm{LN}(\cdot; \mu, \sigma^2) \Leftrightarrow \log(X) \sim \mathcal{N}(\cdot; \mu, \sigma^2)$$

$$E[X] = \exp\left(\mu + \frac{\sigma^2}{2}\right)$$

$$\mathrm{var}[X] = \exp(2\mu + \sigma^2) \cdot [\exp(\sigma^2) - 1]$$

- **Key Result**. If $X \sim \mathrm{LN}(\cdot; \mu_X, \sigma_X^2)$ and $Y \sim \mathrm{LN}(\cdot; \mu_Y, \sigma_Y^2)$ then

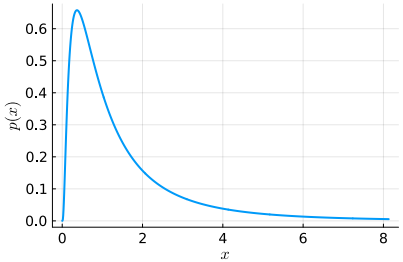
$$X \cdot Y \sim \mathrm{LN}(\cdot; \mu_X + \mu_Y, \sigma_X^2 + \sigma_Y^2)$$

$$\frac{X}{Y} \sim \mathrm{LN}(\cdot; \mu_X - \mu_Y, \sigma_X^2 + \sigma_Y^2)$$

$\log(X) \sim \mathcal{N}(\cdot; \mu_X, \sigma_X^2)$
$\log(Y) \sim \mathcal{N}(\cdot; \mu_Y, \sigma_Y^2)$

If $\log(X) \sim \mathcal{N}(\cdot; \mu_X, \sigma_X^2)$ and $\log(Y) \sim \mathcal{N}(\cdot; \mu_Y, \sigma_Y^2)$
then $\log(X \cdot Y) = \log(X) + \log(Y) \sim \mathcal{N}(\cdot; \mu_X + \mu_Y, \sigma_X^2 + \sigma_Y^2)$

Sir Francis Galton
(1822 – 1911)



Log-Normal Distribution: Representations & Operations

- **Scale-Location Parameters**

$$\mathrm{LN}(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi}\sigma x}\cdot\exp\left(-\frac{(\log(x)-\mu)^2}{2\sigma^2}\right)$$

- **Conversions**

$$\mathrm{LN}(x;\mu,\sigma^2) = \mathrm{LG}\left(x;\frac{\mu}{\sigma^2},\frac{1}{\sigma^2}\right)$$

- **Natural Parameters**

$$\mathrm{LG}(x;\tau,\rho) = \sqrt{\frac{\rho}{2\pi}}\cdot\exp\left(-\frac{\tau^2}{2\rho}\right)\cdot\exp\left((\tau-1)\cdot\log(x)-\rho\cdot\frac{[\log(x)]^2}{2}\right)$$

- **Conversions**

$$\mathrm{LG}(x;\tau,\rho) = \mathrm{LN}\left(x;\frac{\tau}{\rho},\frac{1}{\rho}\right)$$

Two divisions only!

$$\boldsymbol{T}(x) = \begin{bmatrix}\log(x)\\ -\frac{[\log(x)]^2}{2}\end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix}\tau-1\\ \rho\end{bmatrix}$$

- **Theorem (Multiplication).** *Given two Log-Normal distributions* $\mathrm{LG}(x;\tau_1,\rho_1)$ *and* $\mathrm{LG}(x;\tau_2,\rho_2)$

$$\mathrm{LG}(x;\tau_1,\rho_1)\cdot\mathrm{LG}(x;\tau_2,\rho_2) = \mathrm{LG}(x;\tau_1+\tau_2-1,\rho_1+\rho_2)\cdot\mathcal{N}(\mu_1;\mu_2,\sigma_1^2+\sigma_2^2)\cdot\exp\left(\frac{1-2(\tau_1+\tau_2)}{2(\rho_1+\rho_2)}\right)$$

Additive updates!

Correction factor

- **Theorem (Division).** *Given two Log-Normal distributions* $\mathrm{LG}(x;\tau_1,\rho_1)$ *and* $\mathrm{LG}(x;\tau_2,\rho_2)$

$$\frac{\mathrm{LG}(x;\tau_1,\rho_1)}{\mathrm{LG}(x;\tau_2,\rho_2)} = \frac{\mathrm{LG}(x;\tau_1-\tau_2+1,\rho_1-\rho_2)}{\mathcal{N}(\mu_1;\mu_2,\sigma_2^2-\sigma_1^2)}\cdot\frac{\sigma_2^2}{\sigma_2^2-\sigma_1^2}\cdot\exp\left(\frac{2(\tau_1-\tau_2)+1}{2(\rho_1-\rho_2)}\right)$$

Subtractive updates!

Correction factor

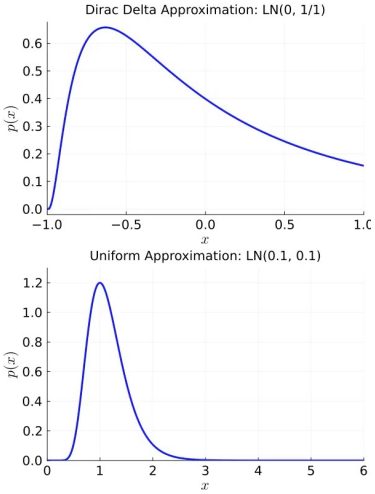
Limit Log-Normal Distributions: Dirac Delta and Uniform

- **Log-Normal Dirac Delta**. The Dirac delta function $\delta(\cdot)$ is defined as

$$\delta_{\mathrm{LN}}(x) = \lim_{\sigma^2 \to 0} \mathrm{LN}(x+1; 0, \sigma^2)$$

- **Log-Normal Uniform**. The Log-Normal uniform $\mathcal{U}_{\mathrm{LN}}(\cdot)$ is defined as

$$\mathcal{U}_{\mathrm{LN}}(x) = \lim_{\sigma^2 \to +\infty} \mathrm{LG}\left(x; 1, \frac{1}{\sigma^2}\right)$$



- **Theorem (Convolution of Log-Normal with Dirac)**. *For any* $\mu, t \in \mathbb{R}$ *and* $\sigma \in \mathbb{R}^+$

$$\int_0^{+\infty} \delta_{\mathrm{LN}}(x-t) \cdot \mathrm{LN}(x; \mu, \sigma^2)\, \mathrm{d}x = \mathrm{LN}(t; \mu, \sigma^2)$$

← Log-Normal density at t

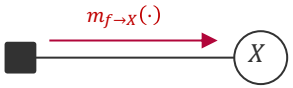
- **Theorem (Product of Log-Normal with Uniform)**. *For any* $\mu \in \mathbb{R}$ *and* $\sigma \in \mathbb{R}^+$

$$\frac{\mathcal{U}_{\mathrm{LN}}(x) \cdot \mathrm{LN}(x; \mu, \sigma^2)}{\int_0^{+\infty} \mathcal{U}_{\mathrm{LN}}(\tilde{x}) \cdot \mathrm{LN}(\tilde{x}; \mu, \sigma^2)\, \mathrm{d}\tilde{x}} = \mathrm{LN}(x; \mu, \sigma^2)$$

← Equivalent to multiplying with 1

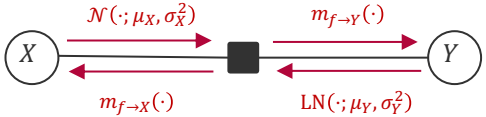
A First Attempt on a Product Factor

Log-Normal Factor $\mathrm{LN}(X; \mu, \sigma^2)$



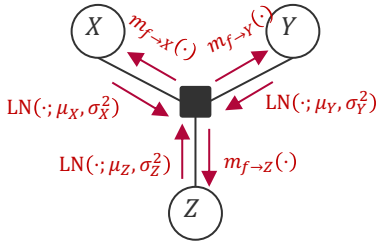
$$m_{f\to X}(x) = \mathrm{LN}(x; \mu, \sigma^2)$$

Exponential Factor $\delta(Y - \exp(X))$



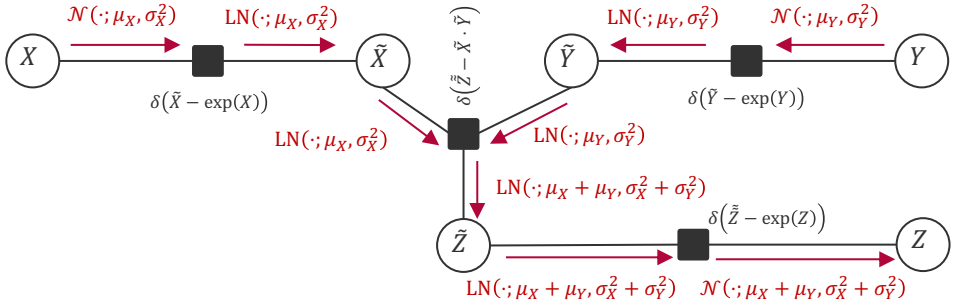
$$m_{f\to Y}(y) = \mathrm{LN}(y; \mu_X, \sigma_X^2)$$
$$m_{f\to X}(x) = \mathcal{N}(x; \mu_Y, \sigma_Y^2)$$

Product Factor $\delta(X \cdot Y - Z)$



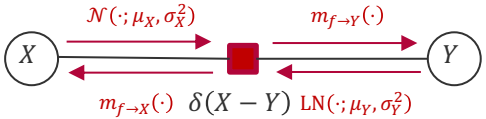
$$m_{f\to Z}(z) = \mathrm{LN}(z;\ \mu_X + \mu_Y, \sigma_X^2 + \sigma_Y^2)$$
$$m_{f\to Y}(y) = \mathrm{LN}(y; \mu_Z - \mu_X, \sigma_Z^2 + \sigma_X^2)$$
$$m_{f\to X}(x) = \mathrm{LN}(x; \mu_Z - \mu_Y, \sigma_Z^2 + \sigma_Y^2)$$

- **Problem**: When putting it all together, we just modelled the sum factor because of the exponential transformation!



Product Factor Through Copying

- **Solution**: Instead of exponentiating the Gaussian inputs, we *copy* the Gaussian (truncating all negative values if necessary)!



$$m_{f\to Y}(y) = \int_{-\infty}^{+\infty} \delta_{\mathcal{N}}(x-y)\cdot\mathcal{N}(x;\mu_X,\sigma_X^2)\,\mathrm{d}x = \mathcal{N}(y;\mu_X,\sigma_X^2)$$

$$m_{f\to X}(x) = \int_{0}^{+\infty} \delta_{\mathrm{LN}}(x-y)\cdot\mathrm{LN}(y;\mu_Y,\sigma_Y^2)\,\mathrm{d}y = \mathrm{LN}(x;\mu_Y,\sigma_Y^2)$$

True messages are **normalized** probability distributions!

$\exp\left(\mu+\frac{\sigma^2}{2}\right)$

$\exp(2\mu+\sigma^2)\cdot[\exp(\sigma^2)-1]$

- **Direct Message Approximation**: We explicitly match the first two moments of $E\left[\widehat{m}_{f\to Y}(\cdot)\right] = E\left[m_{f\to Y}(\cdot)\right]$ and $\mathrm{var}\left[\widehat{m}_{f\to Y}(\cdot)\right] = \mathrm{var}\left[m_{f\to Y}(\cdot)\right]$ (and the same for X)

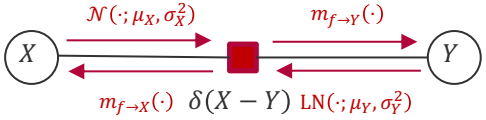
$$\widehat{m}_{f\to Y}(y) = \mathrm{LN}\left(y;\log\left(\frac{\mu_X^2}{\sqrt{\mu_X^2+\sigma_X^2}}\right),\log\left(1+\frac{\sigma_X^2}{\mu_X^2}\right)\right)$$

$$\widehat{m}_{f\to X}(x) = \mathcal{N}\left(x;\exp\left(\mu_Y+\frac{\sigma_Y^2}{2}\right),\exp(2\mu_Y+\sigma_Y^2)\cdot[\exp(\sigma_Y^2)-1]\right)$$

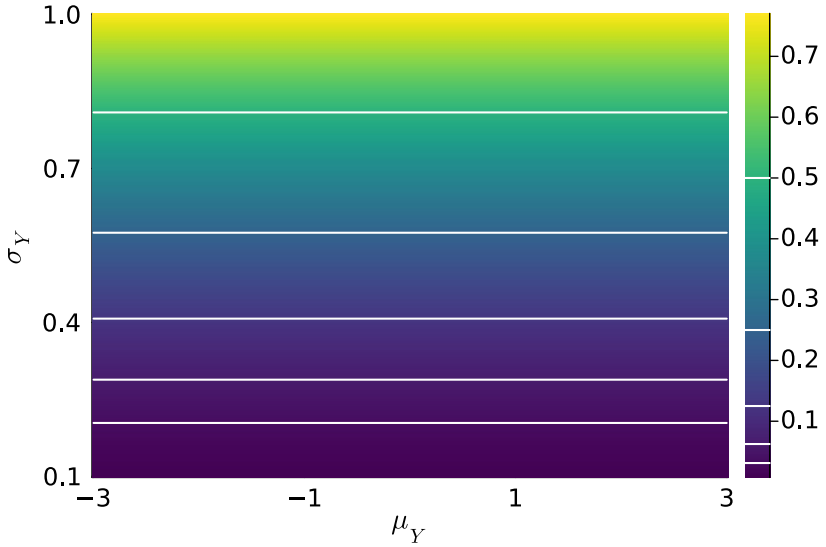
Note that this implies that the approximation only works for $\mu_X > 0$

Copy Factor: Direct Message Approximation Accuracy

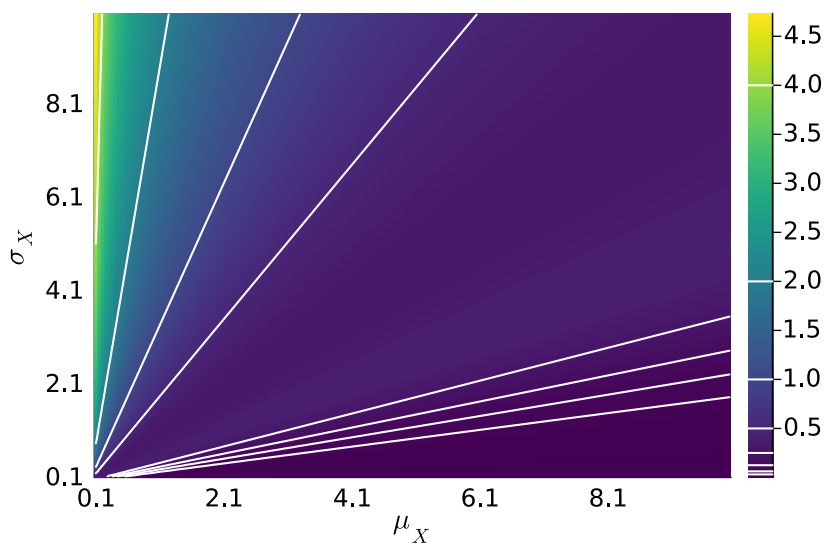
- **Accuracy**: We know that the KL divergence of $\mathrm{KL}[p_X(\cdot), \hat{p}_X(\cdot)]$ is upper-bounded by using $\mathrm{KL}\left[m_{f\to X}(\cdot), \hat{m}_{f\to X}(\cdot)\right]$ (the same is true for Y)



$\mathrm{KL}[\mathrm{LN}(\cdot;\mu_Y,\sigma_Y^2), \hat{m}_{f\to X}(\cdot)]$



$\mathrm{KL}[\mathcal{N}(\cdot;\mu_X,\sigma_X^2), \hat{m}_{f\to Y}(\cdot)]$

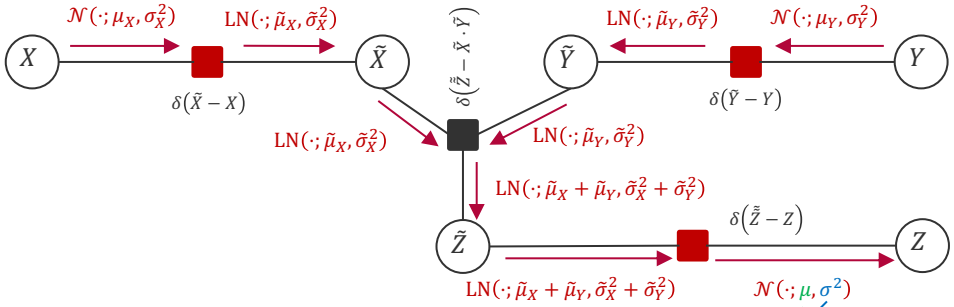


A Second Attempt on a Product Factor

- **Assumption**: Both $\mu_X > 0$ and $\mu_Y > 0$
- **Copy factor on inputs** yields Log-Normals

$$\tilde{\mu}_X = \log\left(\frac{\mu_X^2}{\sqrt{\mu_X^2 + \sigma_X^2}}\right) \qquad \tilde{\sigma}_X^2 = \log\left(1 + \frac{\sigma_X^2}{\mu_X^2}\right)$$

$$\tilde{\mu}_Y = \log\left(\frac{\mu_Y^2}{\sqrt{\mu_Y^2 + \sigma_Y^2}}\right) \qquad \tilde{\sigma}_Y^2 = \log\left(1 + \frac{\sigma_Y^2}{\mu_Y^2}\right)$$



- **Exact Log-Normal product factor** results in

$$\tilde{\mu}_X + \tilde{\mu}_Y = \log\left(\frac{\mu_X^2 \cdot \mu_Y^2}{\sqrt{\mu_X^2 + \sigma_X^2} \cdot \sqrt{\mu_Y^2 + \sigma_Y^2}}\right) \qquad \tilde{\sigma}_X^2 + \tilde{\sigma}_Y^2 = \log\left(\frac{(\mu_X^2 + \sigma_X^2) \cdot (\mu_Y^2 + \sigma_Y^2)}{\mu_X^2 \cdot \mu_Y^2}\right)$$

- **Copy factor on output** finally gives $\mathcal{N}(\cdot;\mu,\sigma^2)$

$$\mu = \exp\left((\tilde{\mu}_X + \tilde{\mu}_Y) + \frac{1}{2}(\tilde{\sigma}_X^2 + \tilde{\sigma}_Y^2)\right) = \exp\left(\log\left(\frac{\mu_X^2 \cdot \mu_Y^2}{\mu_X \cdot \mu_Y}\right)\right) = \mu_X \cdot \mu_Y$$

$$\sigma^2 = \mu^2 \cdot \left[\exp\left((\tilde{\sigma}_X^2 + \tilde{\sigma}_Y^2)\right) - 1\right] = \mu_X^2 \cdot \mu_Y^2 \cdot \left[\frac{(\mu_X^2 + \sigma_X^2) \cdot (\mu_Y^2 + \sigma_Y^2)}{\mu_X^2 \cdot \mu_Y^2} - 1\right] = \mu_X^2 \cdot \sigma_Y^2 + \mu_Y^2 \cdot \sigma_X^2 + \sigma_X^2 \cdot \sigma_Y^2$$

Final Approximate Product Factor

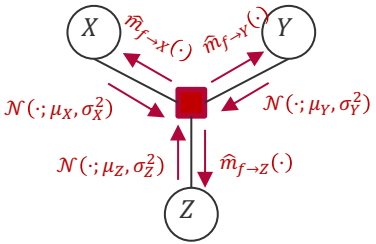
- **Main Idea**: The Copy factor to the Log-Normal distribution (which has mass on the positive axis only!) ensures that the product is *uni-modal*!
  - The closer the means μ_X, μ_Y, or μ_Z are to zero, the worst the approximation!
- **Separating signs and values**: So far, we made the simplifying assumption that $\mu_X > 0$, $\mu_Y > 0$ and $\mu_Z > 0$
  - We deal with the sign by using

$$X \cdot Y = \text{sgn}(X) \cdot |X| \times \text{sgn}(Y) \cdot |Y| = (\text{sgn}(X) \cdot \text{sgn}(Y)) \times (|X| \cdot |Y|)$$

Four cases only which determine if we flip the sign of the mean of the outgoing message

Guaranteed to be well modelled by Log-Normal and flipping means of incoming message

Product Factor $\delta(X \cdot Y - Z)$



$$\widehat{m}_{f\to Z}(z) = \mathcal{N}(z;\ \mu_X \cdot \mu_Y, \mu_X^2 \cdot \sigma_Y^2 + \mu_Y^2 \cdot \sigma_X^2 + \sigma_X^2 \cdot \sigma_Y^2)$$

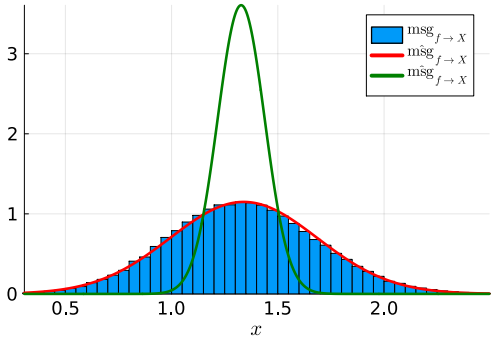
$$\widehat{m}_{f\to Y}(y) = \mathcal{N}\left(y; \left(1 + \frac{\sigma_X^2}{\mu_X^2}\right) \cdot \frac{\mu_Z}{\mu_X}, \left(1 + \frac{\sigma_X^2}{\mu_X^2}\right)^2 \left(\frac{\sigma_Z^2}{\mu_X^2} + \frac{\mu_Z^2}{\mu_X^2} \cdot \frac{\sigma_X^2}{\mu_X^2} + \frac{\sigma_X^2}{\mu_X^2} \cdot \frac{\sigma_Z^2}{\mu_X^2}\right)\right)$$

$$\widehat{m}_{f\to X}(x) = \mathcal{N}\left(x; \left(1 + \frac{\sigma_Y^2}{\mu_Y^2}\right) \cdot \frac{\mu_Z}{\mu_Y}, \left(1 + \frac{\sigma_Y^2}{\mu_Y^2}\right)^2 \left(\frac{\sigma_Z^2}{\mu_Y^2} + \frac{\mu_Z^2}{\mu_Y^2} \cdot \frac{\sigma_Y^2}{\mu_Y^2} + \frac{\sigma_Y^2}{\mu_Y^2} \cdot \frac{\sigma_Z^2}{\mu_Y^2}\right)\right)$$

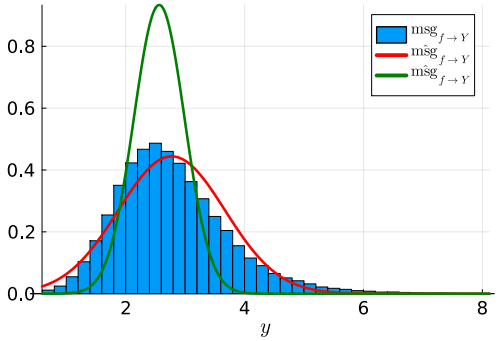
- **Caveat**: We must make sure that none of the two factors X and Y has a mean *exactly* at zero!

Product Factor: Examples

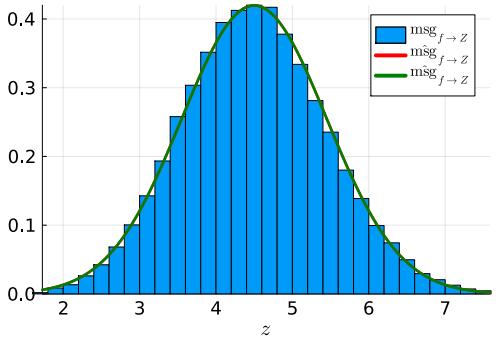
$m_{X\to f}(x) = \mathcal{N}(x; 1.5, 0.3^2)$



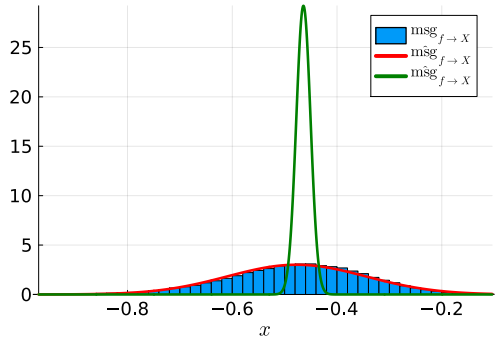
$m_{Y\to f}(y) = \mathcal{N}(y; 3, 0.2^2)$



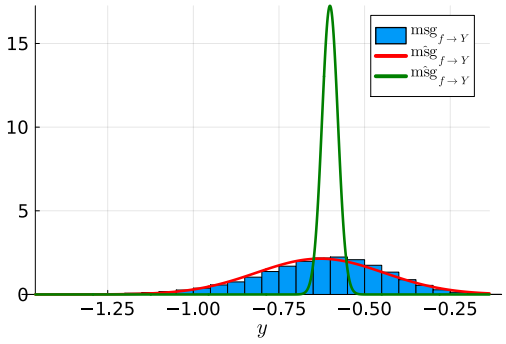
$m_{Z\to f}(z) = \mathcal{N}(z; 4, 1)$



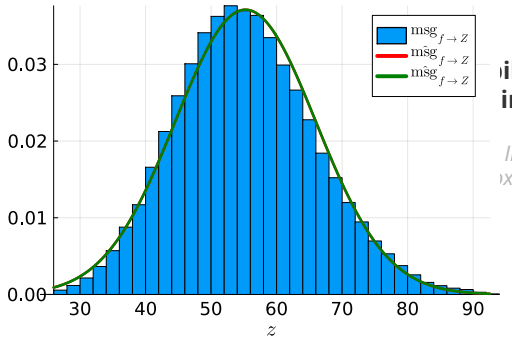
$m_{X\to f}(x) = \mathcal{N}(x; -6.5, 1)$



$m_{Y\to f}(y) = \mathcal{N}(y; -8.5, 1)$



$m_{Z\to f}(z) = \mathcal{N}(z; 4, 1)$



ilistic
ing

Inference:
ximation

Overview

1. Direct Message Approximation
2. **Approximate Messages for Deterministic Functions**
   - Product Factor
   - **Leaky Rectified Linear Unit Factor**
3. Application: Bayesian Neural Networks

Leaky Rectified Linear Unit

- **Leaky Rectified Linear Unit (ReLU)**. *Given $\alpha \in \mathbb{R}^+$ the following function is called the leaky rectified linear unit with factor α*

$$\mathrm{ReLU}_\alpha(x) = \begin{cases} x & x > 0 \\ \alpha \cdot x & x \leq 0 \end{cases}$$

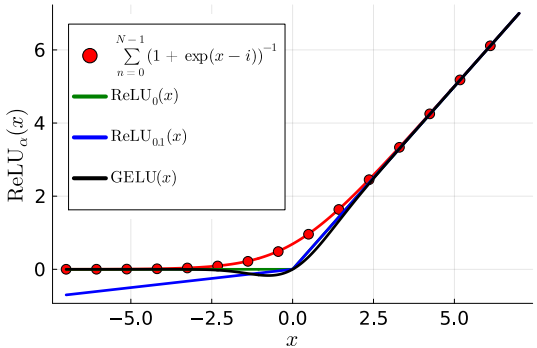
  - ReLU_0 was introduced by Nair & Hinton in 2010 as an approximation for the expected sum of binary activations with probability $(1 + \exp(x - i))^{-1}$ for $i \in \mathbb{N}$ (image models)
  - ReLU_α was introduced in 2013 to avoid vanishing and dying gradients in deep networks

- **Properties**:

$$\frac{\mathrm{d}\,\mathrm{ReLU}_\alpha(x)}{\mathrm{d}x} = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases}$$

Inverse just changes α

$$\mathrm{ReLU}_\alpha^{-1}(y) = \mathrm{ReLU}_{\frac{1}{\alpha}}(y)$$



- ReLU is the precursor to $\mathrm{GELU}(x) = x \cdot \int_{-\infty}^{x} \mathcal{N}(t; 0,1)\,\mathrm{d}t$ which is used in all LLMs!


Vinod Nair (1980 –)


Sir Geoffrey Hinton (1947 –)


Andrew Maas (1987 –)

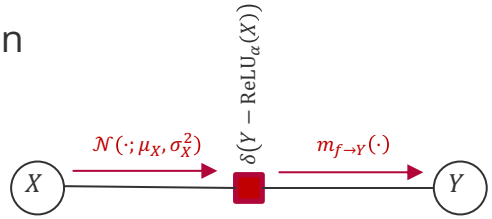

Awni Hannun (1986 –)


Andrew Ng (1976 –)

Leaky Rectified Linear Unit Factor: Approximate Messages

- **Idea**: Since leaky ReLU is a deterministic, monotone and bijective function

$$
\begin{aligned}
m_{f\to Y}(y) &= \int_{-\infty}^{+\infty} \delta_{\mathcal{N}}(\mathrm{ReLU}_\alpha(x) - y) \cdot \mathcal{N}(x; \mu_X, \sigma_X^2)\,\mathrm{d}x \\
&= \int_{-\infty}^{0} \delta_{\mathcal{N}}(\alpha \cdot x - y) \cdot \mathcal{N}(x; \mu_X, \sigma_X^2)\,\mathrm{d}x + \int_{0}^{+\infty} \delta_{\mathcal{N}}(x - y) \cdot \mathcal{N}(x; \mu_X, \sigma_X^2)\,\mathrm{d}x \\
&= \mathbb{I}(y \leq 0) \cdot \frac{1}{\alpha} \cdot \mathcal{N}\left(\frac{y}{\alpha}; \mu_X, \sigma_X^2\right) + \mathbb{I}(y > 0) \cdot \mathcal{N}(y; \mu_X, \sigma_X^2)
\end{aligned}
$$



- **Direct Message Approximation**: We explicitly match the first two moments of $E[\hat{m}_{f\to Y}(\cdot)] = E[m_{f\to Y}(\cdot)]$ and $\mathrm{var}[\hat{m}_{f\to Y}(\cdot)] = \mathrm{var}[m_{f\to Y}(\cdot)]$

$$
\hat{m}_{f\to Y}(y) = \mathcal{N}\left(y; \mu_X A_X + \sigma_X \phi_X (1-\alpha), (\mu_X^2 + \sigma_X^2) B_X + \mu_X \sigma_X \phi_X (1-\alpha^2) - \left(\mu_X A_X + \sigma_X \phi_X (1-\alpha)\right)^2\right)
$$

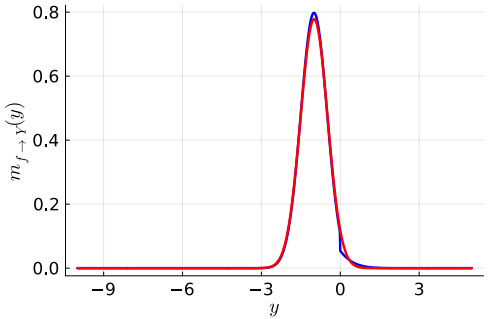
$$
A_X = \alpha + (1-\alpha) P_X \qquad B_X = \alpha^2 + (1-\alpha^2) P_X \qquad \phi_X = \mathcal{N}\left(\frac{\mu_X}{\sigma_X}; 0, 1\right) \qquad P_X = \int_{-\infty}^{\frac{\mu_X}{\sigma_X}} \mathcal{N}(t; 0, 1)\,\mathrm{d}t
$$

- The message $\hat{m}_{f\to X}(\cdot)$ follows from $\delta_{\mathcal{N}}(\mathrm{ReLU}_\alpha(x) - y) = \delta_{\mathcal{N}}\left(\mathrm{ReLU}_{\frac{1}{\alpha}}(y) - x\right)$!

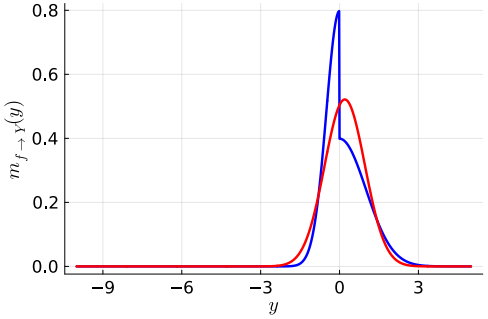
Leaky ReLU Factor Message Approximation: Examples

HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

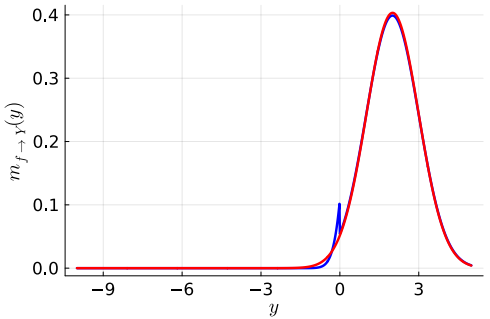
$m_{X\to f}(x) = \mathcal{N}(x; -2,1)$



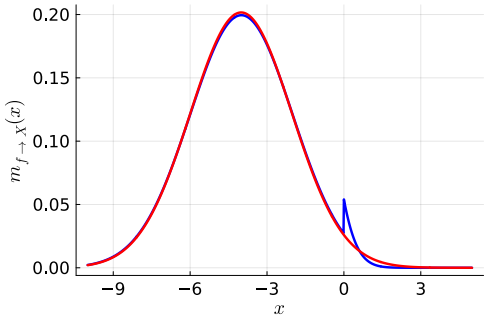
$m_{X\to f}(x) = \mathcal{N}(x; 0,1)$



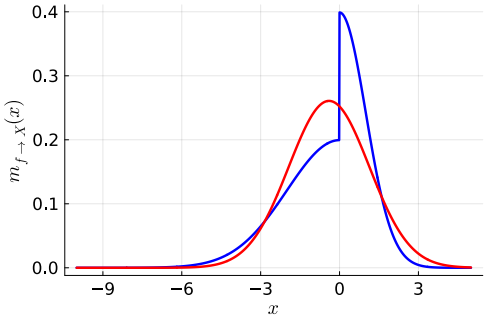
$m_{X\to f}(x) = \mathcal{N}(x; 2,1)$



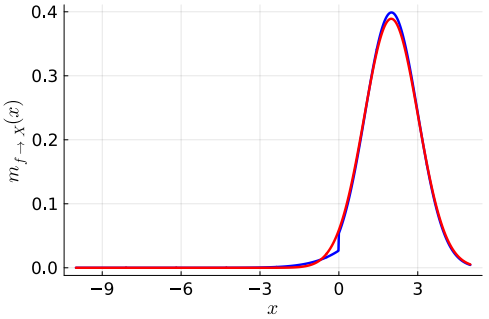
$m_{Y\to f}(y) = \mathcal{N}(y; -2,1)$



$m_{Y\to f}(y) = \mathcal{N}(y; 0,1)$

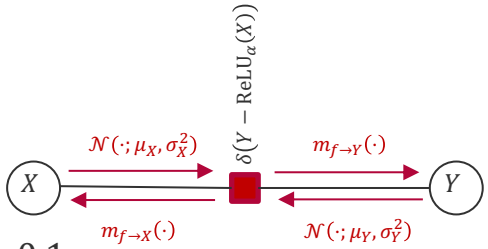


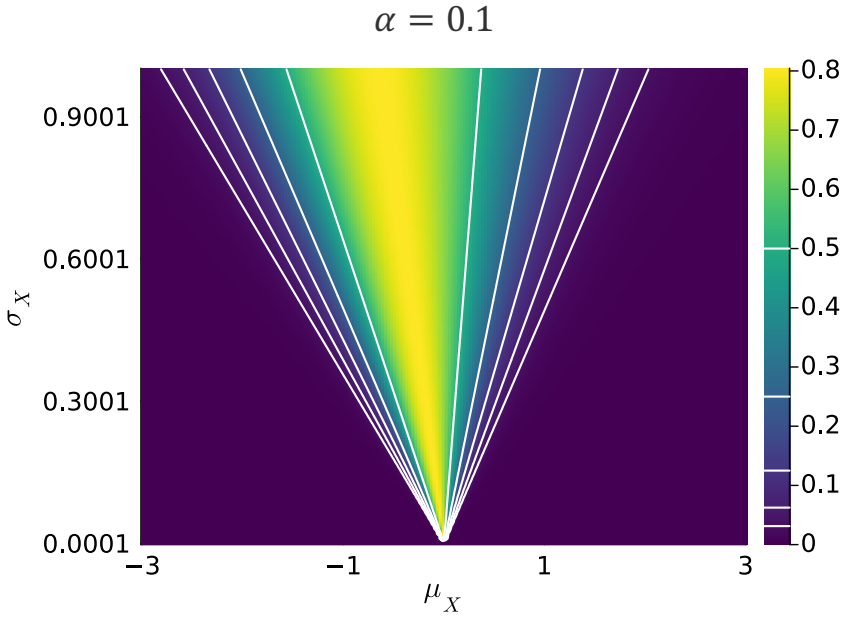
$m_{Y\to f}(y) = \mathcal{N}(y; 2,1)$

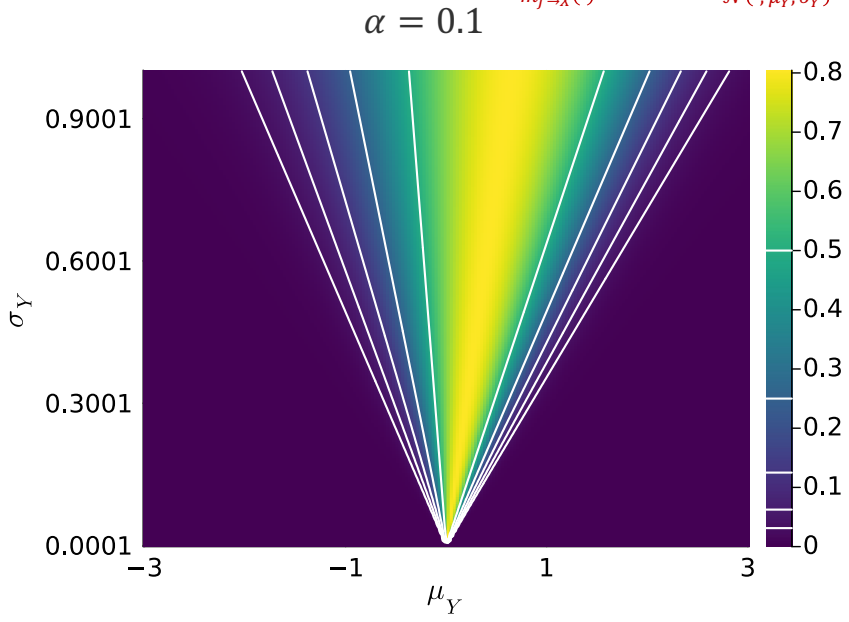


ReLU Factor: Direct Message Approximation Accuracy

- **Accuracy**: We know that the KL divergence of $\mathrm{KL}[p_Y(\cdot), \hat{p}_Y(\cdot)]$ is upper-bounded by $\mathrm{KL}\left[m_{f\to Y}(\cdot), \hat{m}_{f\to Y}(\cdot)\right]$ (the same is true for X)

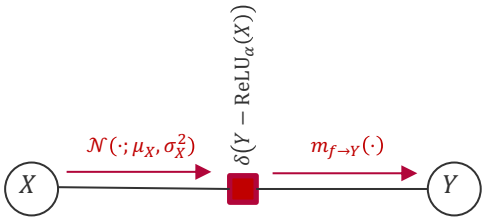




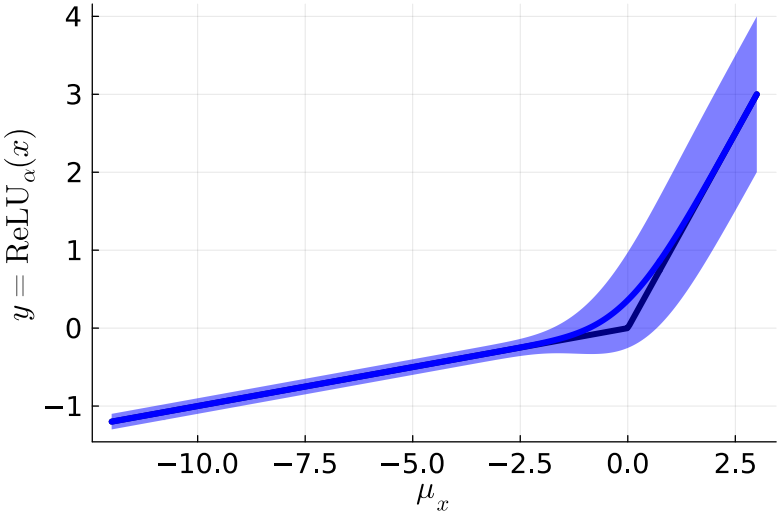


ReLU: Direct Message Approximation and Smoothing

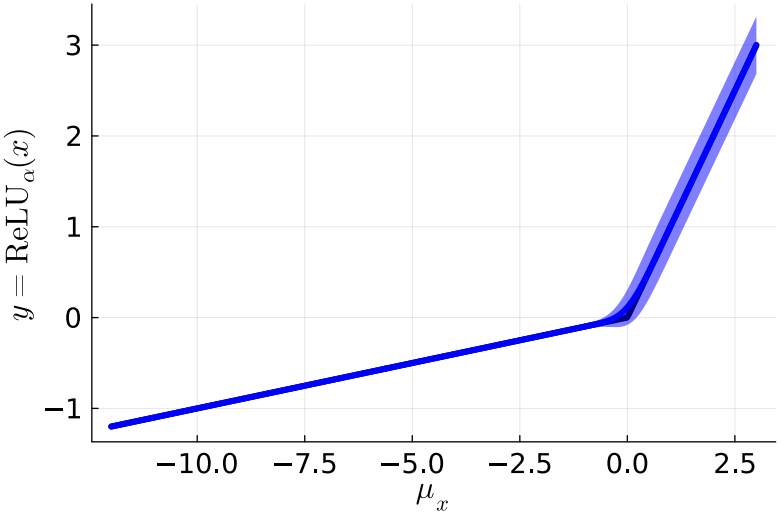
- **Effective Smoothing**: When the incoming message $m_{X \to f}(\cdot)$ has uncertainty $\sigma_X > 0$, the outgoing message has a mean that is effectively a smoothed function of $\mathrm{ReLU}_\alpha(\mu_X)$!



$\sigma_X = 2, \alpha = 0.1$



$\sigma_X = 0.2, \alpha = 0.1$



Overview

1. Direct Message Approximation
2. Approximate Messages for Deterministic Functions
   - Product Factor
   - Leaky Rectified Linear Unit Factor
3. **Application: Bayesian Neural Networks**

Neural Networks

- **Neural Network**: A *neural network* is a layered function approximator $f: \mathcal{X} \to \mathcal{Y}$ trained by adjusting the parameter of the layer functions based on training data $D = \{(x_i, y_i)\} \subseteq \mathcal{X} \times \mathcal{Y}$ so as to match the training data closely

$$f(x) = (g_L \circ g_{L-1} \circ \cdots \circ g_2 \circ g_1)(x)$$

Note that $(g_l \circ g_{l-1})(x) = g_l(g_{l-1}(x))$

- **Classical Layer Functions**: Layer functions are linear mappings with a component-wise non-linearity $h: \mathbb{R} \to \mathbb{R}$ (where $\tilde{\boldsymbol{z}}^{(0)} = \boldsymbol{x}$)

$$g_l\left(\tilde{\boldsymbol{z}}^{(l-1)}\right) = \begin{bmatrix} h\left(\sum_{k=1}^{K_{l-1}} \tilde{z}_k^{(l-1)} \cdot w_{1,k}^{(l)}\right) \\ \vdots \\ h\left(\sum_{k=1}^{K_{l-1}} \tilde{z}_k^{(l-1)} \cdot w_{K_l,k}^{(l)}\right) \end{bmatrix}$$

Parameter of the l-th layer function called *weights*

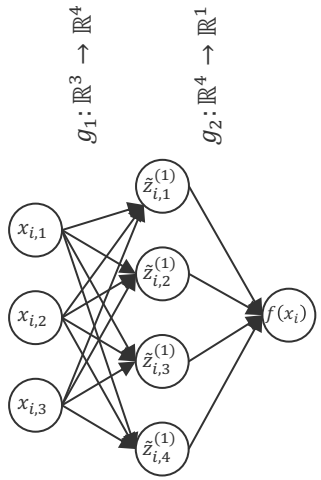
Total number of parameters equals $K_{l-1} \cdot K_l$ per layer and $J = \sum_{l=1}^{L} K_{l-1} \cdot K_l$ in total

- **Neural network learning** is based on the posterior probability $p(\boldsymbol{W}|D)$

Assuming conditional independence and $p(y_i|x_i, \boldsymbol{W}) = p\left(y_i|f(x_i; \boldsymbol{W})\right)$

$$p(\boldsymbol{W}|D) \propto p(D|\boldsymbol{W}) \cdot p(\boldsymbol{W}) = \prod_{i=1}^{n} p(y_i|x_i, \boldsymbol{W}) \cdot p(\boldsymbol{W})$$

Often assuming $p(\boldsymbol{W}) = \prod_j \mathcal{N}(w_j; 0, \lambda^2)$



Neural Networks: Learning Paradigms

HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Classical Neural Network Learning

Efficient and scalable algorithms to approximate the maximum of $p(\boldsymbol{W}|D)$

$$\boldsymbol{W}^* \approx \text{argmax}_{\boldsymbol{W}} \prod_{i=1}^{n} p(y_i|x_i, \boldsymbol{W}) \cdot p(\boldsymbol{W})$$

- **Practice**: The maximum is determined on $\log\big(p(\boldsymbol{W}|D)\big)$

$$\boldsymbol{W}^* \approx \text{argmax}_{\boldsymbol{W}} \sum_{i=1}^{n} \log\big(p(y_i|x_i, \boldsymbol{W})\big) + \sum_{j=1}^{J} \log\Big(p\big(w_j\big)\Big)$$

- **Methodology**: Gradient descent on $p(\boldsymbol{W}|D)$ w.r.t. $\boldsymbol{W}$ using the chain rule of differentiation to compute

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t + \eta \cdot \frac{\partial \sum_{i=1}^{n} \log\big(p(y_i|x_i, \boldsymbol{W})\big) + \sum_{j=1}^{J} \log\Big(p\big(w_j\big)\Big)}{\partial \boldsymbol{W}}\bigg|_{\boldsymbol{W}=\boldsymbol{W}_t}$$

and keep iterating until a local maxima $\boldsymbol{W}^*$ is found

Bayesian Neural Network Learning

Efficient and scalable algorithms to approximate $p(\boldsymbol{W}|D)$ by

$$q \approx \text{argmin}_q \, \text{KL}\left[\prod_{i=1}^{n} p(y_i|x_i, \boldsymbol{W}) \cdot p(\boldsymbol{W}), q(\cdot)\right]$$

($f_i(\boldsymbol{W})$ over $p(y_i|x_i, \boldsymbol{W})$, $f_0(\boldsymbol{W})$ over $p(\boldsymbol{W})$)

- **Practice**: q is product of 1D-Gaussians over $\boldsymbol{W}$

$$q(\boldsymbol{W}) = \prod_{j=1}^{J} \mathcal{N}\big(w_j; \mu_j, \sigma_j^2\big) \propto \prod_{j=1}^{J} \prod_{i=0}^{n} \mathcal{N}\big(w_j; \mu_{i,j}, \sigma_{i,j}^2\big)$$

Approximate message $\hat{m}_{f_i \to w_j}(\cdot)$

- **Methodology**: Message passing to refine the approximations $\mathcal{N}\big(\cdot; \mu_{i,j}, \sigma_{i,j}^2\big)$ of $m_{f_i \to w_j}(\cdot)$
- **Key Insights**:
  - Minimizing KL-divergence results in matching mean $\boldsymbol{\mu}$ and variance $\text{diag}(\boldsymbol{\sigma}^2)$ of $p(\boldsymbol{W}|D)$
  - The maximum of $q(\cdot)$ is attained at $\boldsymbol{\mu} \approx \boldsymbol{W}^*$

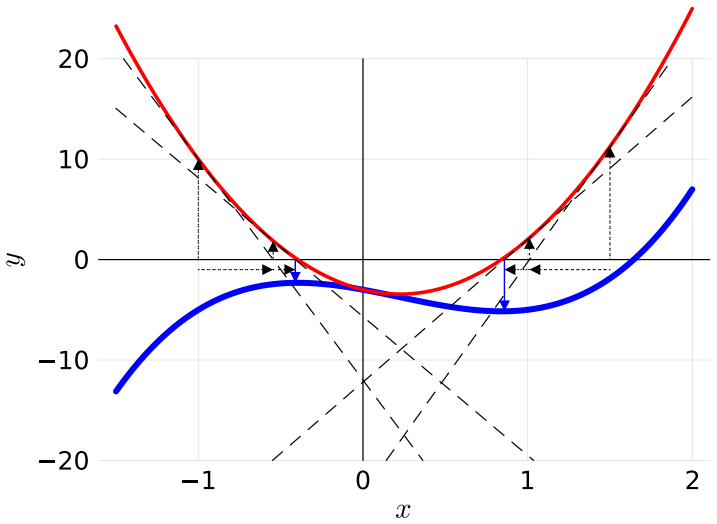
Detour: Newton-Raphson Algorithm

- **Problem**: Find the local extrema of a function $f: \mathbb{R} \to \mathbb{R}$
- **Idea**: Find the zeros of the first derivative f' of the function!
- **Newton-Raphson Algorithm**: Approximate f' at a point x_t with a linear function $g(x) = ax + b$ and find update x_{t+1} such that $g(x_{t+1}) = 0$

$$a = f''(x_t)$$
$$b = f'(x_t) - f''(x_t) \cdot x_t$$

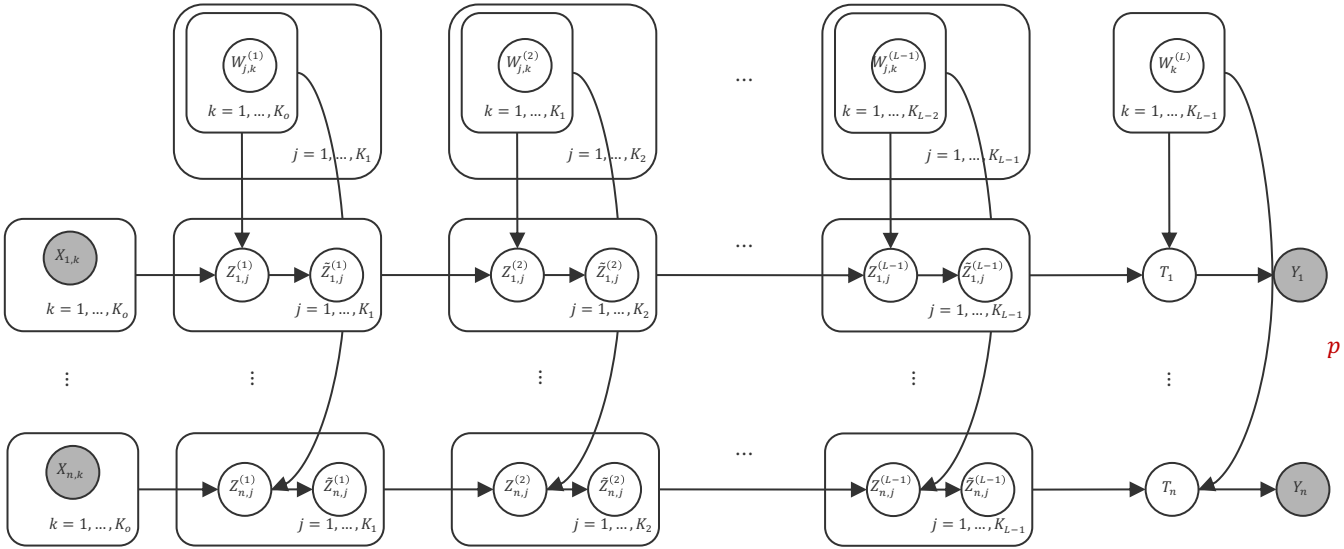
$$x_{t+1} = -\frac{b}{a} = \frac{f''(x_t) \cdot x_t - f'(x_t)}{f''(x_t)}$$

$$x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}$$



Sir Isaac Newton
(1643 – 1727)

Bayesian Neural Networks



Weight Priors

$$p\left(W_{j,k}^{(l)}\right) = \mathcal{N}\left(W_{j,k}^{(l)}; m_{j,k}^{(l)}, \lambda^2\right)$$

Scalar Product Factor

$$p\left(Z_{i,j}^{(1)} | \boldsymbol{W}_j^{(1)}, \boldsymbol{X}_i\right) = \delta\left(\boldsymbol{X}_i^{\mathrm{T}} \boldsymbol{W}_j^{(1)} - Z_{i,j}^{(1)}\right)$$

$$p\left(Z_{i,j}^{(l+1)} | \boldsymbol{W}_j^{(l+1)}, \tilde{\boldsymbol{Z}}_i^{(l)}\right) = \delta\left(\tilde{\boldsymbol{Z}}_i^{(l)} \boldsymbol{W}_j^{(l+1)} - Z_{i,j}^{(l+1)}\right)$$

$$p\left(T_i | \boldsymbol{W}^{(L)}, \tilde{\boldsymbol{Z}}_i^{(L-1)}\right) = \delta\left(\tilde{\boldsymbol{Z}}_i^{(L-1)} \boldsymbol{W}^{(L)} - T_i\right)$$

ReLU Factor

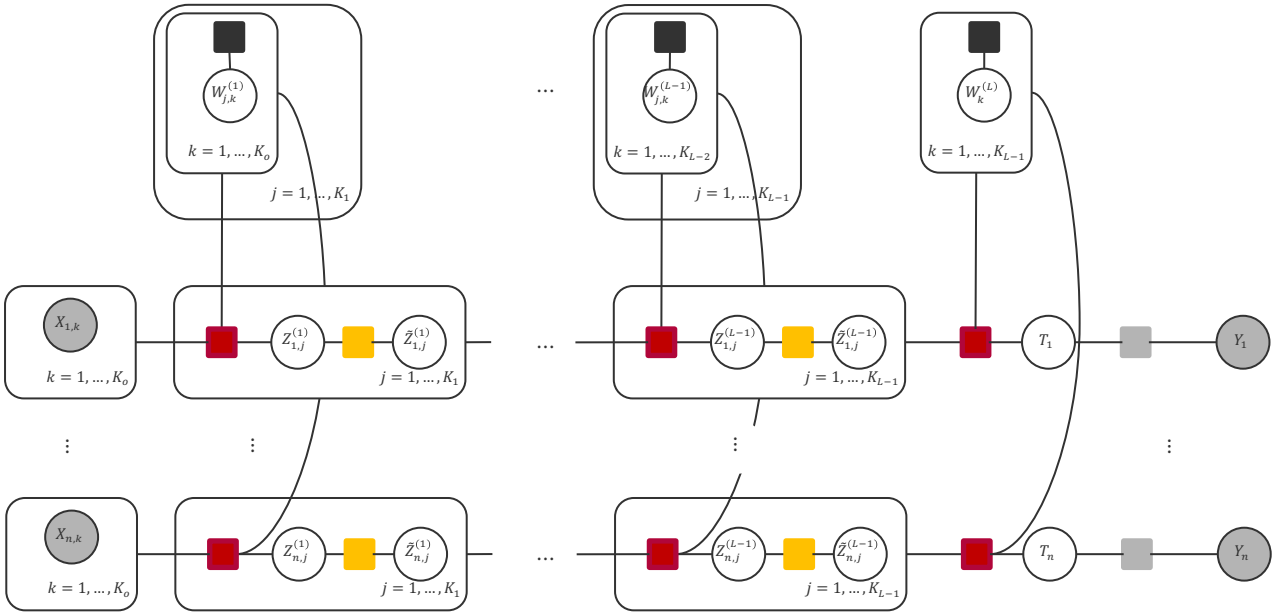
$$p\left(\tilde{Z}_{i,j}^{(l)} | Z_{i,j}^{(l)}\right) = \delta\left(\mathrm{ReLU}_\alpha\left(Z_{i,j}^{(l)}\right) - \tilde{Z}_{i,j}^{(l)}\right)$$

Data Factor

$$p(Y_i | T_i) = \mathcal{N}(Y_i; T_i, \beta^2)$$

- Full model of parameters $\boldsymbol{W}$, latent inner products $\boldsymbol{Z}$, non-linear transformations $\tilde{\boldsymbol{Z}}$, and observed training data $D = \{(\boldsymbol{x}_i, y_i)\}$ is a directed graphical model!

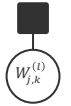
Bayesian Neural Network: Factor Graph



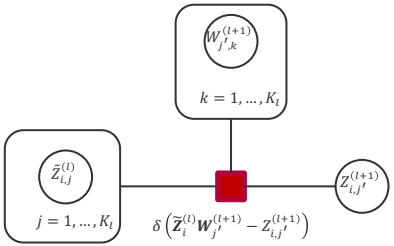
- The factor graph only contains four types of factor nodes
- **Prediction**: Updating messages layer-wise forward from $\boldsymbol{X}_i$ to Y_i
- **Learning**: Updating messages layer-wise backward from Y_i to $\boldsymbol{X}_i$

Weight Prior Factor

$\mathcal{N}\left(W_{j,k}^{(l)}; m_{j,k}^{(l)}, \lambda^2\right)$

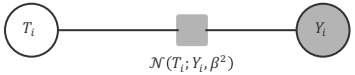


Scalar Product Factor



ReLU Factor

Data Factor



Bayesian Neural Network: Example

Input Transformation $\mathbb{R}^1 \to \mathbb{R}^6$

$$\boldsymbol{x} = \begin{bmatrix} 1 \\ x \\ \exp\left(-\frac{1}{2}(x+1)^2\right) \\ \exp\left(-\frac{1}{2}x^2\right) \\ \exp\left(-\frac{1}{2}(x-1)^2\right) \\ \sin(x) \end{bmatrix}$$

Weight Priors

$$W_{j,k}^{(l)} \sim \mathcal{N}\left(\cdot; m_{j,k}^{(l)}, 1\right)$$

$$m_{j,k}^{(l)} \sim \mathcal{N}(\cdot; 0, 1)$$

Data Model

$$Y_i \sim \mathcal{N}(Y_i; T_i, 1)$$

Architecture

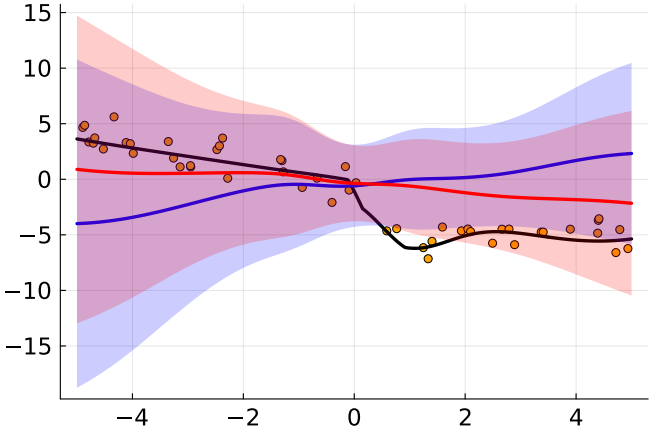
$L = 3$

$K_0 = 6$ $K_2 = 3$

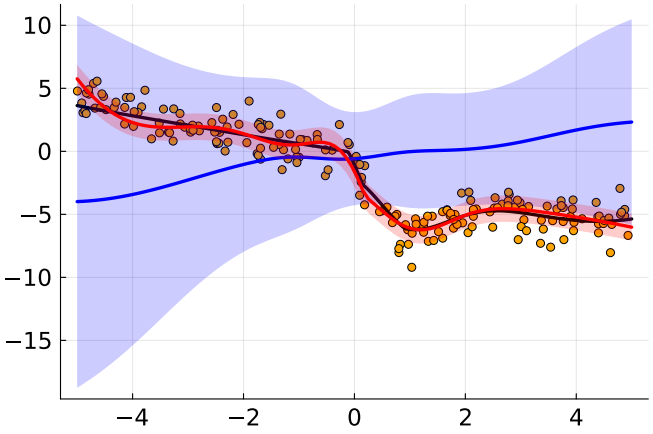
$K_1 = 3$ $K_3 = 1$

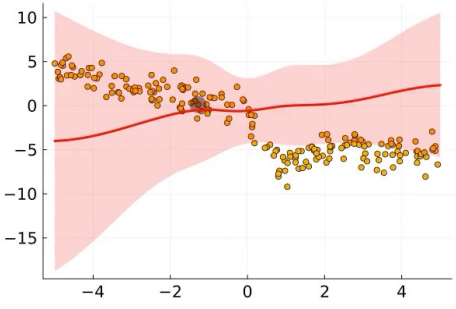
$\alpha = 0.1$

$n = 50$



$n = 200$





Advanced Probabilistic Machine Learning

Unit 7 – Approximate Inference: Direct Message Approximation

Bayesian Neural Networks: Why does it work (so well)?

HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Shun'ichi Amari (甘利 俊) (1936)

- **Observation**: The only direct change to $W_{j,k}^{(l)}$ comes from $\tilde{Z}_{i,k}^{(l-1)}$ and the part of $Z_{i,j}^{(l)}$ that is attributed to this part of the inner product sum (and T_i in the last layer)

$$Z_{i,j}^{(l)} = \sum_{k=1}^{K_l} \tilde{Z}_{i,k}^{(l-1)} \cdot W_{j,k}^{(l)}$$

- **Gradient Descent**: For the product $f(\tilde{z}, w, z) = \lim_{s^2 \to 0} \mathcal{N}(z; \tilde{z} \cdot w, s^2)$, the gradient of the log-factor values has inferior convergence behavior because

$$\frac{\partial \log\big(f(\tilde{z}, w, z)\big)}{\partial w} \propto \tilde{z} \cdot (\tilde{z} \cdot w - z)$$

The gradient misses the second derivative (*natural gradient*) of $\frac{1}{\tilde{z}^2}$ for $w - \frac{z}{\tilde{z}}$ to compute the solution w^* in one step!

- **Direct Message Approximation**: We have convergence in (nearly) one step!

$$m_{f \to W}(\cdot) = \mathcal{N}\left(\cdot; \left(1 + \frac{\sigma_{\tilde{Z}}^2}{\mu_{\tilde{Z}}^2}\right) \cdot \frac{\mu_Z}{\mu_{\tilde{Z}}}, \tau_{f \to W}^{-1}\right) \qquad m_{W \to f}(\cdot) = \mathcal{N}\left(\cdot; \mu_W, \tau_{W \to f}^{-1}\right)$$

$$\left(1 + \frac{\sigma_{\tilde{Z}}^2}{\mu_{\tilde{Z}}^2}\right)^2 \left(\frac{\sigma_Z^2}{\mu_{\tilde{Z}}^2} + \frac{\mu_Z^2}{\mu_{\tilde{Z}}^2} \cdot \frac{\sigma_{\tilde{Z}}^2}{\mu_{\tilde{Z}}^2} + \frac{\sigma_{\tilde{Z}}^2}{\mu_{\tilde{Z}}^2} \cdot \frac{\sigma_Z^2}{\mu_{\tilde{Z}}^2}\right)$$

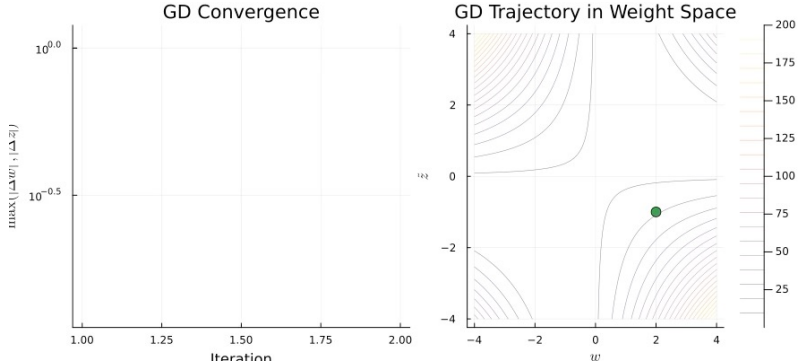
Correct update in mean for $\sigma_{\tilde{Z}}^2 \to 0$!

Message precision act as weighing factors

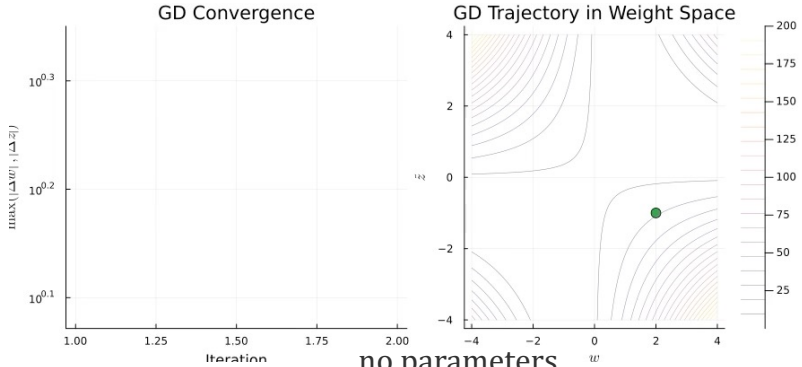
$$p_W(\cdot) \propto \mathcal{N}\left(\cdot; \left(1 + \frac{\sigma_{\tilde{Z}}^2}{\mu_{\tilde{Z}}^2}\right) \cdot \frac{\mu_Z}{\mu_{\tilde{Z}}}, \frac{\tau_{f \to W}}{\tau_{f \to W} + \tau_{W \to f}} + \mu_W \cdot \frac{\tau_{W \to f}}{\tau_{f \to W} + \tau_{W \to f}}, \frac{1}{\tau_{f \to W} + \tau_{W \to f}}\right)$$

Product Factor Convergence Visualization

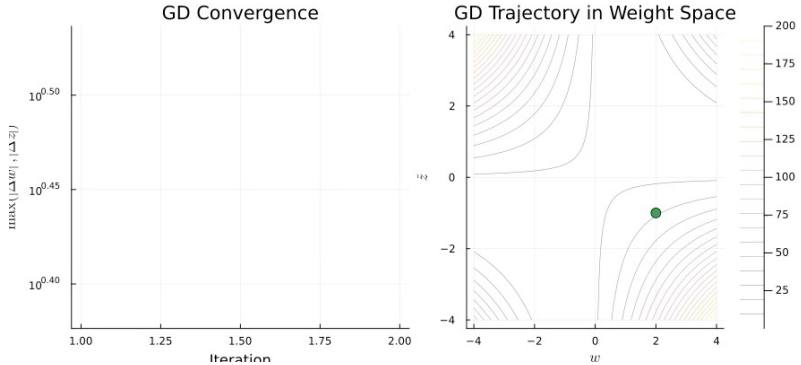
$\eta = 0.01$



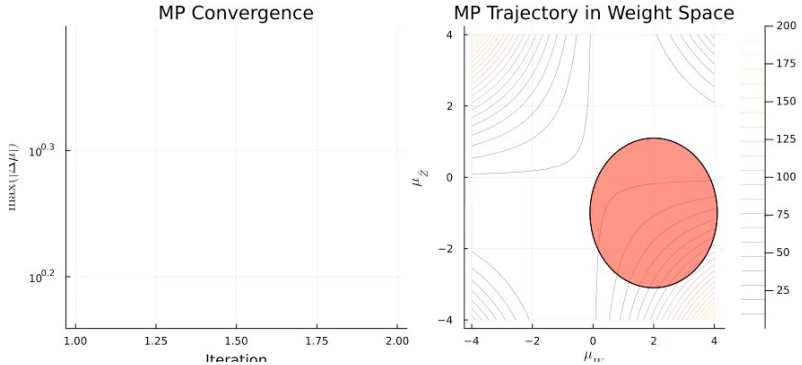
$\eta = 0.1$



$\eta = 0.2$



no parameters



Bayesian Neural Networks: Practical Tips & Tricks

1. **Numerical Stability**: All message update equations must be derived from the natural parameters (τ, ρ) of the incoming messages to the natural parameters (τ, ρ) of the outgoing message and handle uniform distributions
   - Each of the $\#\text{weights} \times \#\text{training examples}$ outgoing message is an approximate of the incremental change to the weight due to the training example. For increasing data, this change becomes zero which equals $\tau = \rho = 0$
2. **Efficient Scalar Product**: We need to update the messages to all elements of each vector in parallel, so the update computation is $\mathcal{O}(K)$ rather than $\mathcal{O}(K^2)$
   - We compute once $c = \sum_{k=1}^{K} \mu_{P_k \to f}$ and $d = \sum_{k=1}^{K} \sigma^2_{P_k \to f}$ for all messages coming from the temporary results P_k of the pairwise products (which is $\mathcal{O}(K)$) and subtract $\mu_{P_k \to f}$ and $\sigma^2_{P_k \to f}$ from c and d when we compute the message to the temporary product P_k
3. **Initialization**: The weight priors cannot be initialized with zero mean because the KL divergence of direct message approximation converges to infinity in this case
   - In the zero-mean case, we would not to break the symmetries of the product factor!

Summary

- **Direct Message Approximation (DMA)**
  - Whenever the factor is a Dirac-delta, the outgoing message is a probability distribution.
  - Approximating the message as a probability distribution with vanishing KL-divergence guarantees that marginals also converge in KL-divergence
- **Approximate Messages for Deterministic Functions**
  - For the copy factor $\delta(X - Y)$ when X is Normal and Y is Log-Normal the direct message approximation via moment-matching is fast, simple and convergent
  - This copy operation allows to compute exact messages for the product factor of the Log-Normal approximations by simple addition or subtraction of means and variances
  - The same technique can also be used for the leaky rectified linear unit (ReLU) factor
- **Bayesian Neural Networks**
  - A *neural network* is a layered function approximator $f: \mathcal{X} \to \mathcal{Y}$ where each layer only uses an inner product and component-wise transfer function such as leaky ReLU.
  - Message passing can be easily implemented with DMA and finds both the mean of the most likely weights together as well as the uncertainties for each weight!
  - It is closely related to gradient descent but handles the product factor more efficient.

HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Thank You!